



MODULE 10

Kotlin Programming Fundamentals

Enterprise Mobile Application Development Bootcamp

Phase 03 | Android Development with Kotlin

Module 10 Roadmap

Six blocks. By the end: comfortable reading and writing idiomatic Kotlin, coming from Swift.

1



Syntax &
Structure

2



Functions &
Lambdas

3



Null Safety

4



OOP &
Extension Fns

5



Coroutines

6



Collections,
Generics & Errors



Kotlin Syntax & Structure

- val vs. var
- Type Inference
- String Templates
- Control Flow: if/when



About Kotlin

- Kotlin was first released in 2016 by the JetBrains team in St. Petersburg
 - It is named after an island near St. Petersburg
- It runs on the JVM, so it is compatible with Java
 - It can also be compiled into JavaScript!
- It became very popular in 2017 when Google chose Kotlin as one of the formal development languages for Android development
- It is now bundled with Android Studio



Goals for Creating the Kotlin Language

- Compactness:
 - Developers wanted a concise language using less code to accomplish tasks
 - Comparisons show that code written in Kotlin is on average 30% shorter than the same code in Java
- Safety:
 - Kotlin's compiler detects many classes of errors that traditionally (in Java) were detected at run-time
 - e.g. Kotlin's compiler can prevent the dreaded NullPointerExceptions
- Interoperability
 - Kotlin works in perfect coordination with Java
 - Use Java standard libraries in Kotlin code
 - Extend Java standard libraries with Kotlin code
 - Mix Kotlin and Java code in the same project
- Less opinionated
 - You can write OOP, but you do not have to!
 - It also supports functional programming and lambda expressions



Kotlin Development Environments

- IntelliJ IDE
 - JetBrains' official IDE
 - Write code, compile, and execute within the IDE
- Kotlin Compiler
 - Separately downloadable
 - Write code in any editor, compile and run at the command line
- Android Studio
 - Another choice for IDE
 - Write code for Android applications, compile, and execute within the IDE
- Online editor
 - Options exist to enter and execute Kotlin code in a web browser



Resources

- Official website:
 - <https://kotlinlang.org/>
- Kotlin playground:
 - <https://try.kotlinlang.org/>
- Weekly Kotlin newsletter:
 - <http://www.kotlinweekly.net/>
- Kotlin for Android documentation
 - <http://developer.android.com/Kotlin/index.html>



val vs. var, and Type Inference

Swift

```
let accountName = "Everyday Checking"  
var balance = 4281.16  
balance -= 42.50
```

Kotlin

```
val accountName = "Everyday Checking"  
var balance = 4281.16  
balance -= 42.50
```

val → let. var → var. Both languages infer the type from the assigned value (String, Double here) without an explicit type annotation, and both let you write one explicitly when it aids clarity: `val balance: Double = 4281.16`.

NOTE: Kotlin is case-sensitive!



Primitive Data Types

- Kotlin data types are similar to Java's
 - Int
 - Long
 - Double
 - Float
 - Short
 - Byte
- There is NO implicit casting
 - Even for a widening conversion!

Code Bite

```
val x = 2          // inferred type
val y: Int         // declared type

val z: Int 2       // declared and inferred
                  // declaration is unnecessary

val w: Int = 4.6    // compile error!

var y: Double
y = x               // compile error!
```



Explicit Casting

- Every primitive data type contains methods for casting to any other type
 - toByte()
 - toInt()
 - toLong()
 - toDouble()
 - toFloat()
 - toShort()
 - toChar()

Code Bite

```
var x = 5
var y: Double

y = x.toDouble()    // compiles
```



String Templates

- Muscle-memory trap:
- Typing \() out of Swift habit inside a Kotlin string produces a literal backslash-paren in the output, not an error
- Which makes this an easy bug to miss.
- Watch for it in early exercises.

Code Bite

```
val name = "Checking"  
val balance = 227.38  
  
println("Account: $name")  
  
println("Double: ${balance * 2}")
```



Branching with if

- **if** is Kotlin's primary syntax for branching

Code Bite

```
if (balance < 100) {  
    print("Account balance is low.")  
}  
  
if (balance > 100) {  
    print("Account balance is good.")  
} else {  
    print("Account balance is low.")  
}
```



Branching with if

- **if** can be used as an expression, not just a statement
- This is Kotlin's version of a ternary operator

Code Bite

```
val status = if (balance < 0) "Overdrawn"  
              else "Current"
```



Control Flow: if and when

- If you chain together multiple **if else** tests together, comparing the same variable, **when** can provide a simpler syntax
- An entire block of code can be executed for a **when** branch

Code Bite

```
val feeDescription = when {  
    balance < 0 -> "Overdraft fee applies"  
    balance < 100 -> "Low balance warning"  
    else -> "No fees"  
}
```



Control Flow: if and when

- As with **if**, **when** can be used as an expression
- It can return a value

Code Bite

```
val accountType = when (typeCode) {  
    1, 2 -> "Checking"  
    3 -> "Savings"  
    else -> "Unknown"  
}
```



Block 1 - Kotlin Syntax & Structure

Skill Check ✓

- What are val and var each equivalent to in Swift?
- What's the Kotlin syntax for embedding an expression (not just a variable) inside a string template?
- Name one capability when has that a traditional switch statement doesn't.



Functions & Lambdas

- Function Syntax
- Default & Named Arguments
- Higher-Order Functions
- Trailing Lambda Syntax



Function Syntax

- Functions begin with the keyword **fun**
- That is followed by the function name
- And then the parameter list
 - Parenthesis are required even if there are no parameters
- The body of the function is enclosed in curly braces ({ })
- If the function returns a value, its return type is listed after the parameters and before the body of the function

Code Bite

```
fun sayHello() {  
    println("Hello Kotlin world!")  
}  
  
fun makeGreeting(name: String): String {  
    return "Hello, $name"  
}
```



Default Arguments

- Function parameters can have default values
 - After the parameter name and data type, assign a value
 - When called, if a value is provided for that parameter that value will be used
 - If no value is provided, the default value will be used
- This makes invoking the function more flexible

Code Bite

```
fun calculateFee(amount: Double,  
                 feeRate: Double = 0.03) {  
    return amount * feeRate  
}  
  
var fee = calculateFee(100.0)  
var specialFee = calculateFee(100.0, 0.05)
```



Named Arguments

- When calling a function, you can assign the parameter values to the parameter names.
 - Instead of passing raw values positionally
- This way, the order of the parameters does not matter
- It can make it easier to invoke a function with many parameters with default values and provide a non-default value for a parameter far down the list

Code Bite

```
fun calculateFee(amount: Double,  
                 feeRate: Double = 0.03) {  
    return amount * feeRate  
}  
  
var fee = calculateFee(amount = 100.0)  
var specialFee = calculateFee(  
    amount = 100.0,  
    feeRate = 0.05)
```



Single Expression Function

- In Kotlin, a **single-expression function** is a concise way to write a function whose body consists of only one expression
- Instead of writing curly braces and an explicit **return** keyword, you assign the expression directly to the function signature using an **=** symbol
 - The **return** is implied

Code Bite

```
fun calculateFee(amount: Double,  
                 feeRate: Double = 0.03) =  
    amount * feeRate
```



Higher-Order Functions

- Functions can be passed to another function as a parameter value
- They also can be returned from a function
- When passing a function using its name, you need to use the function reference operator (::)

Code Bite

```
fun displayAccount(acc: Account) {  
    . . .  
}  
  
loadAccount(252, ::displayAccount)
```



Lambda Expression

- In Kotlin, a **lambda expression** is a short, unnamed block of code that can be treated as a value
- This means you can assign a lambda to a variable, pass it as an argument to a function, or return it from a function
- The data type specification can be inferred
 - In either direction!

Code Bite

```
val sum: (Int, Int) -> Int =  
    { x: Int, y: Int -> x + y }
```

```
val sum = { x: Int, y: Int -> x + y }
```

```
// OR
```

```
val sum: (Int, Int) -> Int =  
    { x, y -> x + y }
```



Single Parameter Shorthand for Lambda

- If a lambda expression has EXACTLY one parameter, you do not have to specify the parameter name
- Instead, you can use the keyword **it** to refer to the parameter

Code Bite

```
val square: (Int) -> Int = { x -> x * x }
```

```
// OR
```

```
val square: (Int) -> Int = { it * it }
```



Trailing Lambda Syntax

- When the last parameter of a function is itself a function, you can pass a lambda expression to it outside the parenthesis

Code Bite

```
loadAccount(242, { acct ->
    println("${acct.id} ${acct.balance}")
})
```

// OR

```
loadAccount(242) { acct ->
    println("${acct.id} ${acct.balance}")
}
```



Block 2 - Functions & Lambdas

Skill Check ✓

- What Swift feature is Kotlin's trailing lambda syntax directly equivalent to?
- What does it refer to inside a single-parameter lambda, and what's its Swift equivalent?
- How do you call a function while skipping some default-valued parameters?



Null Safety

- Nullable Types (?)
- Safe Calls (?.)
- the Elvis Operator (?:)
- Smart Casts
- !! and Why to Avoid It



Nullable Types

- A plain String in Kotlin can NEVER hold null
- The compiler enforces this at compile time, not at runtime
- If the value might be null at runtime, it needs to be declared as a nullable type
 - Simply add a question mark (?) after the data type

Code Bite

```
var nickname: String? = null  
  
nickname = "Rainy Day Fund"
```



Safe Calls

- If a value is nullable, you cannot access its attributes directly using the dot (.) notation
 - Even if the variable has explicitly been assigned a value
- Preceding the dot with a question mark (?.) will compile, however.
- The return value will be nullable itself
 - E.g. the `String.length` property is `Int` but the nullable `String?.length` property is `Int?`

Code Bite

```
val phoneNumber: String? = "555-0142"

// this will not compile
println(phoneNumber.length)

// this will compile
println(phoneNumber?.length)
```



The Elvis Operator

- The Elvis operator (`?:`) is Kotlin's null-coalescing operator
- It lets you provide a backup value to use if the primary value is null at runtime
- It makes a nice complement to the Safe Access operator

Code Bite

```
val displayNumber = phoneNumber ?:  
    "No phone number on file"  
  
val digitCount = phoneNumber?.length ?: 0
```



Smart Casts

- After a null check, Kotlin treats the variable as non-null automatically within that scope
 - No re-binding or unwrapping needed

Code Bite

```
var departmentName: String?  
...  
  
if (departmentName != null) {  
    // no need for safe navigation  
    println(departmentName.uppercase())  
} else {  
    println("Department not assigned")  
}
```



Not Null Assertion (!!)

- The double exclamation point (!!) is the not-null assertion operator
 - "I guarantee this thing is not null"
- It forces an unwrap of the nullable value
- Its use is strongly discouraged
 - Unless it is a narrow, provably safe situation
 - It will throw a NullPointerException at runtime if the value is null

Code Bite

```
val nickname: String? = "Buster"

val forced = nickname!!.length
```



Block 3 - Null Safety

Skill Check ✓

- What's the Kotlin equivalent of Swift's ?? nil-coalescing operator, and what does it do?
- What does a smart cast let you skip that you'd otherwise need after a null check?
- Why is !! treated with the same caution as Swift's force-unwrap operator?



OOP Concepts & Extension Functions

- Classes & Data Classes
- Sealed Classes
- Extension Functions



Basic Class Syntax

- Begin class syntax with the keyword `class`
- Properties are simply variables (or constants) declared at the class level
- Instances are created by executing the class name as if it were a function

Code Bite

```
class Product {  
    var id = 0  
    var name = ""  
    var price = 0.0  
}
```

```
let p1 = Product()  
p1.id = 123  
p1.name = "Widget"  
p1.price = 4.95
```



Constructor Syntax

- Primary constructors are part of the class header block
- Their parameters are listed in parenthesis after the class name
- Their parameters automatically become properties
 - var parameters become readable/writeable properties
 - val parameters become read-only properties
- Primary constructors are just a list of parameters
 - They cannot contain any code or logic

Code Bite

```
class Product(  
    var id: Int,  
    var name: String,  
    var price: Double)  
  
let p1 = Product(1128, "Widget", 24.95)
```



Initializer Blocks

- Initializer blocks exist because primary constructors cannot contain code or logic
- They execute immediately when the primary constructor executes
- You can have multiple initializer blocks
 - They will all execute sequentially from top to bottom

Code Bite

```
class Product(val price: Double) {  
    init {  
        require(price >= 0.0) {  
            "Price cannot be negative!"  
        }  
        println("Product initialized with  
price: $price")  
    }  
}
```



Secondary Constructors

- Secondary constructors are defined inside the class body using the **constructor** keyword
- They are primarily used when you need alternative ways to initialize a class or for interoperability with Java
- Every secondary constructor must delegate to the primary constructor (either directly or indirectly) using the **this** keyword

Code Bite

```
class Employee(val name: String,
               val role: String) {

    constructor(name: String)
        : this(name, "General Staff") {
        println("Secondary constructor called
for $name")
    }

}

val emp1 = Employee("Diana", "Manager")
val emp2 = Employee("Ethan")
```



Data Classes

- A **data class** is a specialized class designed specifically to hold data
- By marking a class with the `data` keyword, the Kotlin compiler automatically generates essential boilerplate methods based on the properties declared in the primary constructor
 - `equals()`
 - `hashCode()`
 - `toString()`
 - `copy()`

Code Bite

```
data class Category(val id: Int,  
                    val name: String)  
  
val c1 = Category(101, "Shoes")  
val c2 = Category(101, "Shoes")  
  
println(c1 == c2)      // true  
// this would be false for ordinary classes
```



Class vs DataClass

Feature	Class	Data Class
Primary Purpose	Encapsulating business logic and system behavior	Storing state, models, and Data Transfer Objects
Structural Equality (equals())	Compares objects by reference unless overridden	Compares objects by value (checks if field contents are identical)
String Output (toString())	Prints a generic memory address placeholder	Prints human-readable property values
Object Cloning (copy())	Not provided out of the box	Is provided to replicate objects with selective modifications
Destructuring Support	No native destructuring features	Generates component() functions allowing properties to be unpacked directly into variables
Constructor Limits	Can be empty or have parameters with or without val/var	Requires at least one parameter in the primary constructor, and all must be marked with val or var
Inheritance Restrictions	Can be modified with open, abstract, or sealed	Cannot be inherited; they are closed and final



Sealed Classes: Kotlin's enum with associated values

- A sealed class has a finite and known number of subclasses
- It is like enum values where you can store data with each enum value
 - And each subclass can store different kinds of data
- The subclasses are typically data classes
 - You would use a data object if there was no associated data

Code Bite

```
sealed class TransferResult {  
    data class Success(  
        val confirmationId: String  
    ) : TransferResult()  
    data class Failure(val reason: String)  
        : TransferResult()  
}
```



When Must be Exhaustive on a Sealed Class

- The **when** operator must handle all cases when switching on a sealed class
- The compiler will fail if there is even one case not handled

Code Bite

```
val message = when (result) {  
    is TransferResult.Success ->  
        "Confirmed: ${result.confirmationId}"  
    is TransferResult.Failure ->  
        "Failed: ${result.reason}"  
}
```



Extension Functions

- In Kotlin, an **extension function** allows you to add new functions to an existing class without modifying its source code or using inheritance
- Once defined, they can be called just like regular member methods using dot notation
- To create an extension function, you prefix the function name with a **receiver type** (the class or data type you want to extend)

Code Bite

```
fun Double.asCurrency(): String {  
    return "%.2f".format(this)  
}  
  
val balance = 4281.16  
println(balance.asCurrency())  
// "$4281.16"
```



Block 4 - OOP Concepts & Extension Functions

Skill Check ✓

- Name two methods a Kotlin data class generates automatically that a Swift struct does not, by default.
- What Swift language feature is a Kotlin sealed class most directly analogous to?
- What does adding fun Double.asCurrency() let you call directly on any Double value?



Coroutines

- suspend Functions
- launch & async
- Structured Concurrency
- Coroutines vs. Swift Concurrency



Suspend Functions

- A suspend function is a function marked with the **suspend** modifier
- It can pause its execution at a suspension point and resume later without blocking the underlying thread
- A suspend function can only be called from another suspend function, or from a coroutine

Code Bite

```
suspend fun fetchAccounts(): List<Account> {  
    val response =  
        httpClient.get("/accounts")  
    return response.body()  
}
```



What is a Coroutine

- Coroutine is a shortening of Cooperative Routine
- Instead of relying on the operating system to forcefully switch between tasks (preemptive multitasking), coroutines voluntarily yield control and cooperate with each other to share resources
- When a coroutine hits a long-running operation, it pauses (suspends) and saves its current execution state to memory
- The thread it was running on is immediately freed up to execute other tasks
- Once the long-running operation completes, the coroutine resumes right where it left off—potentially on an entirely different thread



Launch

- **launch** is a coroutine builder
- It will start a coroutine executing concurrently, without blocking the current thread
- It is referred to as a "fire and forget" operation
 - It runs a block of code asynchronously in the background
 - It does not return any result back to the caller
- It does return a Job object
 - You can use this to wait for the block to finish - `job.join()`
 - You can use this to cancel the block - `job.cancel()`
- **launch** is an extension function of **CoroutineScope**
 - It must always be called inside a valid scope



Use Cases for launch

- Use launch whenever you need to execute a background operation where you do not care about a return value
- Saving data to a local database
- Writing logs
- Sending analytics data
- Triggering a network request that updates a remote state without needing to process a response payload



launch & async: Starting a Coroutine

```
// launch - no return value, like Swift's Task { }
```

```
viewModelScope.launch {  
    val accounts = fetchAccounts()  
    _accountsState.value = accounts  
}
```

```
// async - returns a Deferred<T>, retrieved with .await()
```

```
val deferredAccounts = viewModelScope.async { fetchAccounts() }  
val deferredTransactions = viewModelScope.async { fetchTransactions() }  
val accounts = deferredAccounts.await()  
val transactions = deferredTransactions.await()  
// Both requests run concurrently
```



Structured Concurrency



CoroutineScope

Defines a lifetime coroutines are bound to - conceptually close to a Swift Task's enclosing scope



viewModelScope

An Android-provided scope tied to a ViewModel's own lifetime - covered fully in Module 13



Structured concurrency guarantee

A scope cannot complete until every coroutine it launched completes - the same guarantee Swift's task groups and async let provide



Coroutine Scope

- CoroutineScope is an interface that defines the lifecycle and context for running coroutines
- It serves as a lifetime-restricting container
 - Ensures that every contained asynchronous task is tracked and managed
- If the scope is cancelled, all child coroutines launched inside it are instantly cancelled
- It prevents memory leaks by cleaning up contained tasks when they are no longer needed
- The scope will not finish executing until all its children have completed their work
- If one child fails, the others will be cancelled



Kotlin vs. Swift async

Goal	Kotlin	Swift
Mark as an asynchronous function	<code>suspend fun loadProduct()</code>	<code>func loadProduct async</code>
Call it and wait for its value	<code>loadProduct()</code> inside a coroutine	<code>await loadProduct()</code>
Start two fixed child operations	<code>async { ... }</code> twice in a scope	<code>async let</code> twice
Collect their results	<code>.await()</code> on each <code>Deferred</code>	<code>await</code> the <code>async let</code> values
Start work without a result	<code>launch { ... }</code> in a scope	usually <code>Task { ... }</code> , depending on context



Block 5 - Coroutines

Skill Check ✓

- What's the direct Swift equivalent of Kotlin's suspend keyword?
- When would you reach for async instead of launch to start a coroutine?
- What guarantee does structured concurrency provide, in both Kotlin coroutines and Swift Concurrency?



Collections, Generics & Exception Handling

- List, Map & Set
- map/filter/reduce
- Generics Basics
- try/catch



List vs MutableList

- A List is a generic, ordered collection of elements
- They can contain duplicate elements
- They cannot be structurally modified
 - You cannot add elements, remove elements, or replace elements
- A MutableList supports adding, removing, and updating elements



Sets in Kotlin

- A Set is an unordered collection of unique elements
 - As with lists, Kotlin has read-only sets (Set) and mutable sets (MutableSet)
- Set is actually an interface, with several implementations provided
 - You can check for membership with the `.contains()` method

Implementation	Ordering Behavior	Memory & Performance	How to Create
LinkedHashSet	Preserves insertion order	Slightly higher memory usage	setOf() or mutableSetOf()
HashSet	No order guaranteed	Highly optimized, lowest memory usage	hashSetOf()
TreeSet	Sorted (natural or comparator)	O(log n) operations	sortedSetOf()



Maps

- In Kotlin, a Map is a collection that stores data as key-value pairs.
- Keys must be completely unique
- Each key maps to exactly one value, though values can be duplicated across several keys
- Kotlin provide Maps and MutableMaps

Code Bite

```
val capitalCities = mapOf(  
    "USA" to "Washington D.C.",  
    "UK" to "London",  
    "FR" to "Paris"  
)  
  
val userScores = mutableMapOf(  
    "Alice" to 90,  
    "Bob" to 85  
)  
userScores["Charlie"] = 95  
userScores["Alice"] = 92
```



List, Map & Set

```
val accounts: List<Account> = listOf(checking, savings)           // read-only
val mutableAccounts: MutableList<Account> = mutableListOf(checking)
mutableAccounts.add(savings)

val balancesByAccountId: Map<String, Double> = mapOf(
    "a1" to 4281.16,
    "a2" to 18340.50
)

val uniqueAccountTypes: Set<String> = setOf("Checking", "Savings")
```



Mapping (Transforming) Collections

- The **map** function is a higher-order function used to transform each element of a collection
 - It works on Lists, Sets, and Maps
- It always returns a new List containing the transformed elements
- Variations include:
 - **mapIndexed** - provides the index and value to the transforming function
 - **mapNotNull** - filters out any null results
 - **mapValues**, **mapKeys** - variations for Map collections

Code Bite

```
val numbers = listOf(1, 2, 3)

val tripled = numbers.map { it * 3 }

val customers = mutableListOf<Customer>()
// populate the list

val emails = customers.map { it.email }
```



Filtering Collections

- **filter** is a function used to extract a subset of a collection into a new collection
 - When used on a List or a Set it returns a List
 - When used on a Map it returns a new Map
- Variations include:
 - **filterIndexed** - provides index and value to the filtering function
 - **filterNot** - returns elements NOT matching
 - **filterNotNull** - removes all null values
 - **filterIsInstance<T>** - name says it all
 - **partition** - splits into two collections: the matches and non-matches

Code Bite

```
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8)

val evenNumbers = numbers.filter
    { it % 2 == 0 }

val largeNumbers = numbers.filter
    { it > 5 }
```



Reduce Function

- The **reduce** function aggregates a collection (List, Array, or Set) into a single value
- Its return type is the same as the items in the collection
- It uses the first collection value as the initial aggregate value and starts iterating on the second collection value
- Variations:
 - **reduceOrNull** - returns null instead of crashing on an empty collection
 - **reduceRight** - iterates backwards
 - **runningReduce** - returns a list of all intermediate results along the way

Code Bite

```
val numbers = listOf(1, 2, 3, 4, 5)

val sum = numbers.reduce { acc, num ->
    acc + num
}
```



Fold Function

- Similar to **reduce**, the **fold** function aggregates a collection (List, Array, or Set) into a single value
- Its return type can be different from the collection type
- It requires specification of a starting value, as its iteration starts with the first collection element
- **fold** safely returns the initial value in the event of an empty collection

Code Bite

```
val numbers = listOf(1, 2, 3, 4, 5)

val sum = numbers.fold(0) { acc, num ->
    acc + num
}
```



sumOf Function

- **sumOf** is a specialized function that combines behavior of **map** and **reduce**
- It receives an extractor function (as **map** does) and applies that function to each element
- The resulting values are summed and the result is returned

Code Bite

```
data class Product(val name: String,
    val price: Double, val quantity: Int)

val cart = listOf(
    Product("Laptop", 999.99, 1),
    Product("Keyboard", 80.0, 1),
    Product("Mouse", 49.50, 2)
)

val total = cart.sumOf {
    it.price * it.quantity
}
```



Chaining Methods

- Most of these methods return a List
- Another of these methods can be invoked on the returned List
- Rather than storing each result in separate variables, they can be chained together

Code Bite

```
val balances = accounts.map { it.balance }

val overdrafts = accounts.filter {
    it.balance < 0 }

val totalBalance = accounts.fold(0.0) {
    sum, account -> sum + account.balance }

// chained:
val overdraftTotal = accounts
    .filter { it.balance < 0 }
    .sumOf { it.balance }
```



Generic Classes

- Generics allow you to create classes, interfaces, and functions that operate on any data type while maintaining compile-time type safety
- Generic classes declare a type parameter inside angle brackets
- The data type is specified when an instance of the class is created

Code Bite

```
class Box<T>(val content: T)

val intBox = Box<Int>(42)
val stringBox = Box<String>("Hello")
```



Generic Functions

- Generic functions declare a type parameter before the function name
- The type can be used on parameters, the return type, and even on variables and constants within the function
 - Usually one or more parameters are of that type
- The data type is inferred when the function is invoked

Code Bite

```
fun <T> printItem(item: T) {  
    println(item)  
}
```

```
val first: Int = 42  
println(first)
```

```
val second: String = "Hello"  
println(second)
```



Generic Constraints

- With total flexibility, you have very little ability to do anything with the generic parameter
 - e.g. you can't use a length property or an `append()` method because not all data types have those attributes
- Generic constraints restrict the range of data types that can be used, in order to give you some knowledge of the attributes the data types have
- Single constraints use a colon inside the angle brackets
- Multiple constraints use a **where** clause at the end of the declaration

Code Bite

```
class Box<T: Number>(val value: T)

val intBox = Box(10)      // ok
val stringBox = Box("Hi") // compile error

fun <T> resetAndRecord(item: T)
    where T: Clearable, T: Loggable {
    item.clear()
    item.log()
}
```



Exception Handling

- In Kotlin, try-catch blocks are used to handle exceptions and prevent the application from crashing
- Optionally, a **finally** block can be added
 - It will run unconditionally, as soon as either the **try** or the **catch** block have completed
- Kotlin does not require functions to declare which exceptions they throw
 - No throws keyword on the signature
- Any function can throw any exception at any time

Code Bite

```
try {  
    val result = 10 / 0    // throws  
} catch (e: ArithmeticException) {  
    println("Cannot divide by zero:  
            ${e.message}")  
}  
finally {  
    println("This will always execute")  
}
```



Try is an Expression

- Because try is an expression, it returns a value
- It can be used to conditionally assign values to a variable or constant

Code Bite

```
fun parseIntOrNull(input: String): Int? {  
    val number = try {  
        input.toInt()  
    } catch (e: NumberFormatException) {  
        null  
    }  
  
    return number  
}
```



Block 6 - Collections, Generics & Exception Handling

Skill Check ✓

- What's the difference between `listOf(...)` and `mutableListOf(...)`?
- Name the Kotlin collection function equivalent to Swift's `filter`.
- Unlike Swift's `throws` keyword, how does Kotlin handle declaring which exceptions a function might throw?



PNC Mobile: Kotlin Fundamentals Lab

Skill Check ✓

- Model Account as a Kotlin data class.
- Write a sealed class TransferResult with Success and Failure cases, and a when that handles both exhaustively.
- Write an extension function Double.asCurrency() for formatted balance display.
- Write a suspend function that simulates fetching accounts with a delay(), called from a coroutine.
- Use filter, map, and sumOf to compute total balance and flag overdraft accounts from a List<Account>.



Module 10 Complete

You can now read and write idiomatic Kotlin: syntax and control flow, functions and lambdas, null safety, data classes and sealed classes, extension functions, coroutines, and collections. Every one of these maps to something you already know from Swift - the vocabulary changed; the engineering judgment didn't.